

Bulletproofs: applications

July 31, 2025

Distributed Lab

 zkdl-camp.github.io

 github.com/ZKDL-Camp



Plan

- 1 Introduction
- 2 IPA polynomial commitment scheme
- 3 Range proofs
- 4 Arithmetic circuits
- 5 Security, efficiency, and open questions

Introduction
●○○○

IPA polynomial commitment scheme
○○○

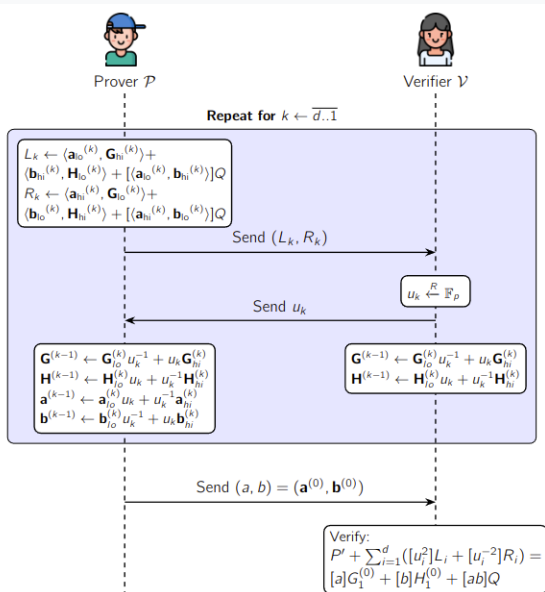
Range proofs
○○○○○○○○○

Arithmetic circuits
○○○○○

Security, efficiency, and open questions
○○○○○

Introduction

Inner-product argument: illustration



Recap: inner-product argument

- **Goal:** Prove $\langle a, b \rangle = c$ with logarithmic proof size
- **Commitment:** $P' = \langle a, G \rangle + \langle b, H \rangle + [\langle a, b \rangle]Q$
- Protocol recursively compresses vectors, sending L_k, R_k at each step
- **Final check:** $P' + \sum ([u_i^2]L_i + [u_i^{-2}]R_i) = [a]G + [b]H + [ab]Q$

Key property

Proof size is $2 \log_2 n$ group elements + 2 scalars. Protocol is *knowledge sound* and *perfect complete* but not *zero-knowledge*.

Idea

We could provide *zero-knowledge* directly to *inner-product argument* construction or use **zk-mul** protocol for outer construction.

Recap: zk-mul

Consider relation $R_{mul} = \{(\perp; l(x), r(x), t(x)) \mid t(x) = l(x)r(x)\}$ where $l(x) = a + s_L x$, $r(x) = b + s_R x$, $t(x) = l(x)r(x)$. Protocol **zk-mul** is defined as follows:

- Prover computes and sends to $\mathcal{V}$ commitments to $l(x), r(x), t(x)$:

$$A = [a]G + [b]H + [\alpha]B$$

$$T_0 = [ab]G + [\tau_0]B$$

$$S = [s_L]G + [s_R]H + [\beta]B$$

$$T_1 = [s_L + s_R]G + [\tau_1]B$$

$$T_2 = [s_L s_R]G + [\tau_2]B$$

- Verifier draws random challenge $u \in \mathbb{F}_p$ and sends it to prover
- Prover evaluates and sends to Verifier $(l_u, r_u, t_u, \alpha_u, \tau_u)$:

$$l_u = l(u), r_u = r(u), t_u = l_u \cdot r_u, \alpha_u = \alpha + \beta u, \tau_u = \tau_0 + \tau_1 u + \tau_2 u^2$$

- Verifier checks: $A + [u]S \stackrel{?}{=} [l_u]G + [r_u]H + [\alpha_u]B$,
 $[t_u]G + [\tau_u]B \stackrel{?}{=} T_0 + [u]T_1 + [u^2]T_2, t_u \stackrel{?}{=} l_u r_u$

IPA polynomial commitment scheme

Recap: Polynomial commitments

- Polynomial commitment scheme

$$\mathcal{C} = (\text{Setup}, \text{Commit}, \text{Open}, \text{VerifyOpen})$$

allows to commit to a polynomial $f(x) = \sum_{i=0}^{n-1} a_i x^i$ and prove its evaluation at some point.

- **Applications:** SNARKs compiled with IOP + polynomial commitment scheme framework (e.g., Halo, Nova, Spartan, Plonk)
- **Desirable properties:** sublinear size, efficient, no trusted setup

Example

One famous example is the **KZG** polynomial commitment scheme, which uses bilinear pairings and requires a trusted setup.

IPA polynomial commitment

Let $f(x) = \sum_{i=0}^{n-1} a_i x^i$ be a polynomial of degree $n - 1 = 2^d - 1$.

The **non-hiding IPA polynomial commitment scheme** $\mathcal{C}_{ip} = (\text{Setup}, \text{Commit}, \text{Open}, \text{VerifyOpen})$ is defined as follows:

- **Setup** returns independent generators $G = (G_1, \dots, G_n)$.
- **Commit** returns $\text{Com}(f) = \langle f, G \rangle$ where $f = (a_0, \dots, a_{n-1})$
- **Open** given evaluation point $u \in \mathbb{F}_p$ computes $u^n = (1, u, u^2, \dots, u^{n-1})$, obtains $f(u) = \langle f, u^n \rangle$ and runs *inner-product argument* Π_{ip} non-interactively setting

$$a = f, b = u^n, P = \text{Com}(f), c = f(u)$$

to produce an evaluation proof π_{ip}

- **VerifyOpen** given evaluation point $u \in \mathbb{F}_p$ and commitment $\text{Com}(f)$ validates proof π_{ip} running the non-interactive verifier $\mathcal{V}$ of *inner-product argument*.

Introduction
○○○○

IPA polynomial commitment scheme
○○○

Range proofs
●○○○○○○○○

Arithmetic circuits
○○○○○

Security, efficiency, and open questions
○○○○○

Range proofs

Range proofs: motivation

- **Goal:** Prove $v \in [0, 2^n)$ without revealing v :

$$\mathcal{R}_{rp} = \{(G, B, V, n; v, \gamma) \mid V = [v]G + [\gamma]B, v \in [0, 2^n)\}$$

- **Applications:** Confidential transactions (e.g., Monero, Mumblewimble), other privacy-preserving protocols.
- **Idea:** Prove $v = \sum_{i=0}^{n-1} v_i 2^i$ and $\forall i \in 0..n-1 : v_i \in \{0, 1\}$

Naive approach

One could prove $v = \sum_{i=0}^{n-1} v_i 2^i$ and $\forall i \in 0..n-1 : v_i \in \{0, 1\}$ using Σ -protocols, but this would be inefficient due to linear proof.

Compiling range proof into inner-product

Firstly, write v in base-2 representation: $v = \sum_{i=0}^{\lfloor \log_2 v \rfloor} 2^i v_i$ and $a_L = (v_0, v_1, \dots, v_{n-1})$ be the vector of bits padded with zeroes to length n , define $a_R = a_L - 1^n$ so the range validation that v lays in $[0, 2^n)$ implies two checks:

- The following inner-product equality holds: $\langle a_L, 2^n \rangle = v$
- Each bit v_i must be either 0 or 1:

$$a_L - a_R - 1^n = 0^n$$

$$a_L \circ a_R = 0^n$$

Example

Let $a_L = (1, 0, 1, 0)$, $a_R = (0, -1, 0, -1)$, then $a_L \circ a_R = (0, 0, 0, 0)$

Compiling range proof into inner-product

This two checks imply verification that some vector is zero vector, for that we use some challenge $y \in \mathbb{F}_p$ and check inner-product equalities

$$\langle a_L \circ a_R, y^n \rangle = 0 \text{ and } \langle a_L - a_R - 1^n, y^n \rangle = 0$$

This checks are sound because the prover doesn't know challenge y in advance. So we must combine three inner-product checks:

1. $\langle a_L, 2^n \rangle = v$
2. $\langle a_L, a_R \circ y^n \rangle = 0$
3. $\langle a_L - a_R - 1^n, y^n \rangle = 0$

into one soundly summing up with powers of other challenge $z \in \mathbb{F}_p$:

$$z^2 \cdot \langle a_L, 2^n \rangle + z \cdot \langle a_L - a_R - 1^n, y^n \rangle + \langle a_L, a_R \circ y^n \rangle = z^2 v$$

Compiling range proof into inner-product

Using some dark linear algebra wizardry we could combine the three inner-product checks into a single one inner-product check:

$$\langle a_L - z \cdot 1^n, z^2 \cdot 2^n + z \cdot y^n + a_R \circ y^n \rangle = z^2 v + \delta(y, z)$$

Where $\delta(y, z)$ could easily be computed by verifier:

$$\delta(y, z) = (z - z^2) \langle 1^n, y^n \rangle - z^3 \langle 1^n, 2^n \rangle$$

Now it's time to bring out **zk-mul** for inner-products!

Firstly, construct the blinding polynomials for a_L and a_R :

$$a'_L \leftarrow a_L + s_L x \quad a'_R \leftarrow a_R + s_R x$$

Compute polynomials $l(x) = l_0 + l_1 x$, $r(x) = r_0 + r_1 x$:

$$l(x) = a'_L - z \cdot 1^n = (a_L + s_L x) - z \cdot 1^n = a_L - z \cdot 1^n + s_L x$$

$$\begin{aligned} r(x) &= z^2 \cdot 2^n + z \cdot y^n + a'_R \circ y^n = z^2 \cdot 2^n + z \cdot y^n + (a_R + s_R x) \circ y^n \\ &= z^2 \cdot 2^n + z \cdot y^n + a_R \circ y^n + s_R \circ y^n x \end{aligned}$$

Compiling range proof into inner-product

$$t(x) = \langle l(x), r(x) \rangle = t_0 + t_1x + t_2x^2$$

Now $\mathcal{P}$ needs to apply **zk-mul** for proving:

$$t_0 = \langle a_L - z \cdot 1^n, z^2 \cdot 2^n + z \cdot y^n + a_R \circ y^n \rangle = z^2 v + \delta(y, z)$$

Note: $\mathcal{V}$ could compute commitment $Com(t_0)$ using $V = Com(v)$

Remark

We couldn't apply raw **zk-mul** as l_0 depends on verifier-provided challenges, instead $\mathcal{P}$ firstly commits to a_L, a_R and blinders s_L, s_R , obtains challenges y, z from $\mathcal{V}$ and computes rest of the commitments.

During verification phase $\mathcal{V}$ should adjust commitments to $l(x), r(x)$ by himself using homomorphic properties of Pedersen commitment scheme.

Range proofs: building the protocol

- Setup returns independent generators $G, H \in \mathbb{G}^n$
- Prover does bit decomposition of v : $a_L \leftarrow v, b_L \leftarrow a_L - 1^n$,
chooses blinding terms $s_L, s_R \in \mathbb{F}_p^n, \alpha, \beta \in \mathbb{F}_p$, sends commitments:

$$A = \langle a_L, G \rangle + \langle b_L, H \rangle + [\alpha]B \quad S = \langle s_L, G \rangle + \langle s_R, H \rangle + [\beta]B$$

- Verifier $\mathcal{V}$ samples challenges $y, z \xleftarrow{R} \mathbb{F}_p$ and sends them to $\mathcal{P}$
- Prover $\mathcal{P}$ reconstructs polynomials $l(x), r(x), t(x)$:

$$l(x) = a_L - z \cdot 1^n + s_L x$$

$$r(x) = z^2 \cdot 2^n + z \cdot y^n + a_R \circ y^n + s_R \circ y^n x$$

$$t(x) = \langle l(x), r(x) \rangle = t_0 + t_1 x + t_2 x^2$$

$$t_0 = \langle a_L - z \cdot 1^n, z^2 \cdot 2^n + z \cdot y^n + a_R \circ y^n \rangle = z^2 v + \delta(y, z)$$

$$t_1 = \langle a_L - z \cdot 1^n, y^n \circ s_R \rangle + \langle y^n \circ (a_R + z \cdot 1^n) + z^2 \cdot 2^n, s_L \rangle$$

$$t_2 = \langle s_L, y^n \circ s_R \rangle$$

Range proofs: proving

- Prover $\mathcal{P}$ draws blinding factors $\tau_1, \tau_2 \xleftarrow{R} \mathbb{F}_p$ and sends to $\mathcal{V}$ commitments for coefficients of $t(x)$:

$$T_1 = [t_1]G + [\tau_1]B$$

$$T_2 = [t_2]G + [\tau_2]B$$

Note: prover does not have to send commitment to t_0 as it's the inner-product we want to prove and it could be computed from high-level commitment V .

- Verifier $\mathcal{V}$ samples and sends to $\mathcal{P}$ evaluation point $u \xleftarrow{R} \mathbb{F}_p$
- Prover $\mathcal{P}$ evaluates polynomials at u :

$$l_u = l(u)$$

$$\alpha_u = \alpha + \beta u$$

$$r_u = r(u)$$

$$\tau_u = z^2\gamma + \tau_1 u + \tau_2 u^2$$

$$t_u = t(u) = t_0 + t_1 u + t_2 u^2$$

and sends $(l_u, r_u, t_u, \alpha_u, \tau_u)$ to $\mathcal{V}$.

Range proofs: verification

- Verifier $\mathcal{V}$ checks:

$$\begin{aligned}
 A + [u]S + \langle -z \cdot 1^n, G \rangle + \langle z \cdot y^n + z^2 \cdot 2^n, y^{-n} \circ H \rangle \\
 &\stackrel{?}{=} \langle l_u, G \rangle + \langle r_u, y^{-n} \circ H \rangle + [\alpha_u]B \\
 [t_u]G + [\tau_u]B &\stackrel{?}{=} [z^2]V + [\delta(y, z)]G + [u]T_1 + [u^2]T_2 \\
 t_u &\stackrel{?}{=} \langle l_u r_u \rangle
 \end{aligned}$$

Remark

To provide logarithmic size-proof instead of sending l_u, r_u parties could run an inner-product argument **IPA** on inputs $(G, y^{-n} \circ H, P, t_u; l_u, r_u)$ where:

$$P = A + [u]S + \langle -z \cdot 1^n, G \rangle + \langle z \cdot y^n + z^2 \cdot 2^n, y^{-n} \circ H \rangle - [\alpha_u]B$$

Introduction
○○○○

IPA polynomial commitment scheme
○○○

Range proofs
○○○○○○○○○

Arithmetic circuits
●○○○○

Security, efficiency, and open questions
○○○○○

Arithmetic circuits

Proving arithmetic circuit satisfiability

- **Goal:** Prove knowledge of w s.t. $C(w) = 0$ for circuit C
- Encode as R1CS: $Aw \circ Bw = Cw$
- Each constraint is a multiplication of linear forms

Bulletproofs for circuits: Approach

- Prover commits to witness w
- Encodes all constraints as inner-products
- Uses IPA protocol to prove all constraints in zero-knowledge

Batching

All constraints can be aggregated into a single inner-product argument

Bulletproofs for circuits: Example

- Circuit: $w_1 \cdot w_2 = w_3$, $w_3 + w_1 = 7$
- Encode as R1CS, commit to w
- Prove in zero-knowledge using bulletproofs

Bulletproofs for circuits: Performance

- Proof size: $2 \log_2 n + O(1)$ group elements (n = number of constraints)
- No trusted setup
- Efficient for small/medium circuits
- Verification time linear in n

Security, efficiency, and open questions

Security and efficiency

- **Security:** Discrete log assumption, no trusted setup
- **Efficiency:** Logarithmic proof size, fast prover for small n
- **Limitations:** Verification time linear in n

Open questions and extensions

- Can we get sublinear verification?
- Can we combine with SNARKs for better efficiency?
- Applications: privacy, blockchains, verifiable computation
- Extensions: batch proofs, recursive proofs, Halo/Nova

Summary

- Bulletproofs: efficient, no trusted setup, versatile
- IPA polynomial commitment: core primitive
- Range proofs and circuit proofs: practical applications
- Still active area of research!

Thank you!

Questions?